

EXERCISES TO COMPLETE

Introduction

Inheritance is a feature of Object-Oriented Programming languages such as Java, which gives a subclass all of the fields and methods from its superclass. Like all language features, it's important to understand both what inheritance does, and how it can be used to solve problems.

The main advantage of inheritance is **code reuse** — solving a problem once, and then reusing the same code to solve many instances of that problem.

Exercise 1 – Harvard Referencing

As an example, in this lab we will design a class hierarchy to model publications which might be referenced in academic writing.

Every `Publication` has some basic details: a list of authors, a title, and the year it was published. However, there are several different kinds of publication, with slightly different additional details:

- A `Book` has a publisher.
- A `JournalArticle` has a `journalName`, volume and issue numbers, and page numbers `pageFrom` and `pageTo`.
- A `WebPage` has a `url`, and a `dateAccessed`.

One solution to this problem could be to declare an interface `Publication` with methods `getTitle`, `getAuthors` and `getYear`. The three classes `Book`, `JournalArticle` and `WebPage` would all implement the `Publication` interface.

Discuss with your class tutor why although this solution works it's not the best way to implement the code for the Harvard Referencing application.

A better solution is to make `Publication` a class and declare the fields and the method implementations, which the other three classes (`Book`, `JournalArticle` and `WebPage`) can inherit.

Download the file `HarvardReferencing.zip` that contains the java source files for this project and use the information provided in the Lecture Notes `Class Relationships` and `Methods Overriding` and `Solving problems using inheritance` to complete the implementation of the rest of the code.

Use the steps below as a guide to put the code together.

CMP5332 Lab 9 -Sheet (Solving Problems Using Inheritance)

Implement the three publication classes (`Book`, `JournalArticle` and `WebPage`). A skeleton code is given in the zipped file with detailed comments to assist you in implementing these classes.

Note that all three subclasses extends the `Publication` class and needs to explicitly call the `super` constructor, to pass the `authors`, `title` and `year` parameters. Other than this, there should be no duplicated code; each subclass silently inherits the fields and methods for the basic details.

Generating the Harvard references

Our task now is to generate references in the Harvard format. Since this format is different for different publication types, each subclass will have its own implementation of the `harvardReference` method.

The correct reference format for a book is:

Knuth, D. (1968) *The Art of Computer Programming: Fundamental algorithms*. Addison-Wesley.

authors (*year*) *title*. *publisher*.

The correct reference format for a journal article is:

Hristova, M., Misra, A., Rutter, M. and Mercuri, R. (2003) Identifying and correcting Java programming errors for introductory computer science students. *ACM SIGCSE Bulletin*, 35(1), pp. 153–156.

authors (*year*) *title*. *journalName*, *volume*(*issue*), pp. *pageFrom-pageTo*.

The correct reference format for a web page is:

Oracle (2016) *The Java Tutorials*. Available at: <https://docs.oracle.com/javase/tutorial/> [Accessed 30 November 2018].

authors (*year*) *title*. Available at: *url* [Accessed *dateAccessed*].

The reference format always begins with the **authors**, **year** and **title**. This behaviour should be common to all classes, so it can be implemented in the superclass. Also, since a publication may have multiple authors, the names must be joined together into one string.

CMP5332 Lab 9 -Sheet (Solving Problems Using Inheritance)

The `harvardReference()` method returns a `String` representing the

- Authors name
- Year, and
- Publication Title

In the format as described above.

The Harvard referencing format requires author names to be separated by **commas**, or the word **“and”** for the last two names. This is done using a loop as described below that returns a `String` of authors names.

```
public String authorNames() {
    String nameString = "";

    int secondLast = authors.size() - 2;
    for(int i = 0; i < authors.size(); i++) {
        nameString += authors.get(i).fullName();
        if(i < secondLast) {
            nameString += ", ";
        } else if(i == secondLast) {
            nameString += " and ";
        }
    }

    return nameString;
}
```

By writing the `authorNames()` method separately, the `harvardReference` method is written in a straightforward way.

The `Book` subclass should override the `harvardReference` method by adding the book's publisher to the reference.

The `JournalArticle` subclass overrides the `harvardReference` method and by the additional details to include in a journal article reference. Including `journalName`, `volume`, `issue`, `pageFrom`, `pageTo`.

Finally, the `WebPage` subclass must also override the `harvardReference` method.

Authors

An author could simply be modelled as a string for their name. However, this would limit the usefulness of our model, because an author like Donald Knuth should appear in a reference as “Knuth, D.”, but in a citation only as “Knuth”.

There are also two kinds of authors: individual authors (such as Donald Knuth) and institutional authors (such as Oracle). Oracle should be “Oracle” in both references and citations.

Therefore it makes sense for `IndividualAuthor` and `InstitutionalAuthor` to be two types of `Author`. However, there are no common implementation details; an individual has a last name and their initials, whereas an institution has only a full name.

In this case since there is no code to be reused via inheritance, it may be better to use an interface instead of a superclass:

```
public interface Author {  
    public String fullName();  
  
    public String citeName();  
}
```

These will be implemented differently by the two classes `IndividualAuthor` and `InstitutionalAuthor`.

The decision between using an interface or using a superclass generally depends on whether there is any benefit from inheritance. If a lot of code can potentially be reused by inheritance, then designing a class hierarchy is a good idea.

However, a class can implement many interfaces, but it can only inherit from one class — so using an interface is less restrictive, and generally better when there is nothing to inherit.